# Semester Technical Report - Parallel Construction of BVH

Jakub Votrubec
CTU FIT, NI-MCC 2025/26

## 1 Problem Definition and Sequential Solution

### 1.1 Problem Definition

This work focuses on the optimization and parallelization of the algorithm for constructing the BVH (Bounding Volume Hierarchy) acceleration data structure. This structure is crucial for the efficient execution of raytracing. The Raytracer was developed within the NI-PG1 course. The goal is to minimize the tree construction time (Build time) on multi-core x86 architecture processors using OpenMP and vectorization.

### 1.2 Sequential Algorithm and Implementation

The starting point is the sequential implementation of BVH. The project implements the `BvhNaive` variant and the more optimized `BvhSeq` / `BvhSeq2` variants (`BvhSeq` was found too complicated and slower and was rewritten into `BvhSeq2`).

The construction algorithm works using the *Top-Down* method:

1. Start with the root node containing all scene triangles.
2. For each node, find the best split plane.
3. The SAH (Surface Area Heuristic) is used to determine the split cost:

$$Cost = C_t + \frac{S_L}{S_{parent}} \cdot N_L \cdot C_i + \frac{S_R}{S_{parent}} \cdot N_R \cdot C_i \quad (1)$$

4. If the split cost is lower than the leaf cost and the maximum BVH tree parameters are not reached, the triangles are split, and the construction is recursively called for the left and right children.

Where $C_t$ is traversal cost, $C_i$ is the cost of an intersection with a node and $S$ is the surface area of the node bounding box (combination of all triangle boudning boxes in one node).

The `BvhSeq2` implementation uses the *Binning* technique, which reduces the number of tested split planes, thereby speeding up SAH evaluation compared to the naive approach where all triangle edges are tested [1].

The number of bins used is 16, Maximum depth of the BVH tree is 32 and the maximum numnber of triangles in a leaf is 4.

**BvhNode** structure was designed to be memory efficient (32 bytes) and aligned for better cache utilization. It is used by all the implementation versions. `Vec3` is a custom structure of 3 single precision floating point numbers used for calculations. In practice `size_t` is used instead of `int`, that could be improved to truly fit inside 32 bytes of memory.

```
struct BvhNode {
    Vec3 aabb_min;
    Vec3 aabb_max;
    int t_begin; // or left child (right child is t_begin+1)
    int t_count;
};
```

**AABB** is a datastructure that holds the minimum and maximum 3D points of a bounding box. It also has a few helpful methods like `surface_area()` and `expand()` used in binning and other calculations.

```
struct AABB {
    Vec3 min;
    Vec3 max;
    float surface_area();
    void expand(AABB other);
    void expand(Vec point);
};
```

### 1.3 Testing data

For testing data I have chosen three model scenes: Bunny, Dragon, Hairball and Powerplant, with approximately 70k, 870k, 2 880k and 12 701k triangles respectively. These scenes/models were chosen to test the implementations of varying sizes and shapes. All of the models and enclosed in a CornellBox. You can see the renders in figure 1.

## 2 Parallelization and Optimization (Vectorization and OpenMP)

This chapter describes the algorithm modifications for parallel processing on the CPU (implementations `BvhPar`, `BvhPar2`).

### 2.1 Compiler flags

As per usual the compiler is a lot smarter than I am when it comes to optimization, so the lowest hanging fruits were optimized by it. When creating `BvhVec` I used the compiler with flags `-ftree-vectorize -fopt-info-vec -fopt-info-vec-missed` to get hints of were to adjust my code and where to use SoA.

When I was developing `BvhVec2` the usage of these flags was minimal due to my choice to use vector in a different approach. This meant that the compiler might not have been able to optimize my code further in some parts.

For the whole code base I used these compiler flags: `-O3 -ffast-math -funroll-loops`. Enabeling the agressive optimization decresed the build time significantly, without hindering the build quality (the result was the same BVH tree).

For SIMD instructions and other intrinsic families I used the `-march=native` compiler flag, so it could choose the

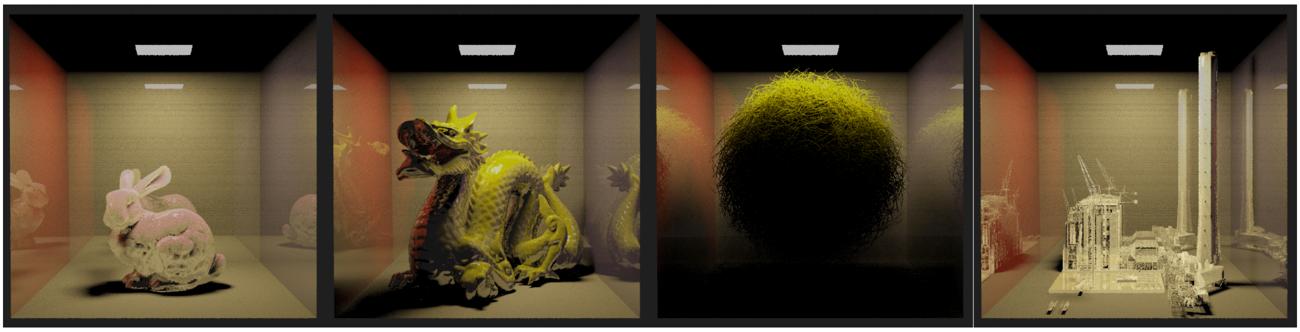

Figure 1: Rendered images of the test scenes: Bunny, Dragon, Hairball, and Powerplant, each enclosed in a CornellBox.

vectorization family, that would fit the architecture the code was compiled on the best.

### 2.2 Additional Data Structures:

In this section I will describe the data structures used in the vectorized and parallel version.

**vAABB** is an alternative AABB structure, that is used for holding the minimum and maximum 3D points in SIMD 128-bit vectors.

```
struct vAABB {
    __m128 aabb_min;
    __m128 aabb_max;
};
```

**Job** is a helpfull datastructure to hold data needed for starting up a parallel task on a separate thread.

```
struct Job {
    size_t node_idx;
    int depth;
    AABB cb;
};
```

### 2.3 Vectorization

As part of the optimization, the possibility of vectorization (AVX/SSE SIMD instructions) was explored in the `BvhVec` and `BvhVec2` implementations.

**BvhVec** was the first variant implemneted. It uses SIMD instructions from the AVX familily with 256-bit registers (holding f32x8). When implementing I first wanted to lean of the compiler to do the vectorization for me, but for any more sophisticated loops it was unable to identify the safety and I was forced to vectorize thses loop manully.

The structure of arrays (SoA) pattern was used to increase cache locality - reduce the amount of cache misses and increase the amount of continuous memory reading. This was to a great effect when it came to paralelizing the binning phase of the algorithm. However when it came to the initial triangle bounding box and centroid calculation the restructuralization of the data greatly overshadowed the cost reduction from vectorization, so much so, that it almost negated the performance gain in the binning phase.

The vectors always contain subsequent values of data. For example holding 8 position Xes of the min position of a bounding box. Resulting in 8 bounding boxes being loaded, calculated and stored at once.

**BvhVec2** uses a different approach. The crucial difference is in the usage of 128-bit vector registers from the SIMD famlily (holding f32x4). Instead of holding subsesquent values of data, the each vector holds one 3D point (x, y, z, 0), with the last element not being used. This changed the SoA to a diffent, more readable and easier to work with format. On the other hand it forced me to do all the vectorization manually, since this is not the "natural" way of vector usage.

The initial calculation of triangle bounds and centroids is now much faster, however the binning and findnig a best split calculations are basically on par with the sequential version. The centroid bounds recalculation si slower then the sequential version, so I have erased it from the implementation.

You can see all the gathered data in the 1 table. Moving forward to the paralelization using OpenMP and multiple threads, I have chosen to extend the `BvhVec2` version.

### 2.4 Parallelization

For parallelization, the OpenMP library was chosen. Due to the recursive nature of the BVH construction algorithm the task parallelizm paradime lends itself as the obvious choice. The problem is, that initially we have a lot less branches to parallelize and each processes a lot of triangles. To mitigate this problem I split the parallelization into a horizontal and vertical phase.

**Horizontal paralelization**

**Vertical paralelization**

## 3 Results

### 3.1 Measured Values

Measurements were performed on a TODO processor with parameters:

```
TODO cluster parameters.
```

## 4 Conclusion

## 5 Literature


1. Ingo Wald: *On fast Construction of SAH-based Bounding Volume Hierarchies*, 2007.

2. OpenMP 5.0 Documentation.

Table 1: Build times (parts and total) for BvhSeq2, BvhVec and BvhVec2 for the Bunny, Dragon and Hairball scenes. The unaccounted for time is the overhead of other operations, that are the same for all the implementations

| **Scene** | **BVH types** | **Init [ms]** | **Binning [ms]** | **Paritioning [ms]** | **Total time [ms]** |
|---|---|---|---|---|---|
| Bunny | BvhSeq2 | 1.93 | 7.39 | 7.8 | 31.8 |
| Bunny | BvhVec | 6.96 | 2.49 | 8.42 | 37.3 |
| Bunny | BvhVec2 | 1.54 | 8.24 | 7.75 | 32.57 |
| Dragon | BvhSeq2 | 26.82 | 101.94 | 97.69 | 408.43 |
| Dragon | BvhVec | 91.34 | 32.05 | 106.44 | 487.05 |
| Dragon | BvhVec2 | 22.54 | 109.13 | 97.28 | 417.21 |
| Hairball | BvhSeq2 | 85.08 | 407.91 | 305.42 | 1396.2 |
| Hairball | BvhVec | 301.13 | 123.75 | 386.7 | 1817.1 |
| Hairball | BvhVec2 | 71.48 | 416.1 | 308.42 | 1395.56 |


3. C++20 Standard library documentation

4. NI-MCC and NI-PG1 Lectures.